

DATA CUT

Symphonie des données en flux continu

Rapport technique

Master 1 Intelligence Artificielle — École 89

Intervenant : Najib AL AWAR

Étudiant : Amaury Guindon

Année : 2025–2026

Sommaire

- 1. Introduction & contexte
- 2. Architecture de la plateforme
- 3. Schéma de base de données
- 4. Pipeline de données
- 5. Extraction de features
- 6. Versioning des datasets
- 7. Entraînement du modèle
- 8. Visualisation & supervision
- 9. Recommandation client par IA
- 10. Bonnes pratiques & gouvernance
- 11. Conclusion & perspectives

1. Introduction & contexte

1.1 Présentation du projet

DataCut est une plateforme web de gestion de données conçue dans le cadre du projet « Symphonie des données en flux continu ». Elle s'inscrit dans le contexte fictif d'un barbershop haut de gamme souhaitant exploiter ses photos de réalisations pour entraîner un modèle de classification de styles de coupe.

L'objectif central est de mettre en œuvre un cycle de vie complet des données : de l'ingestion d'images brutes jusqu'à l'alimentation continue d'un modèle d'intelligence artificielle, en passant par la labélisation, le versioning et l'export structuré.

1.2 Problématique

Comment concevoir une infrastructure capable de collecter, structurer, labéliser et versionner des images, tout en garantissant la traçabilité complète du flux de données vers un modèle d'apprentissage automatique ?

1.3 Périmètre fonctionnel

- Backoffice (admin) : gestion du pipeline, labélisation, création et export de datasets versionnés, supervision via graphiques.
- Interface client : recommandation personnalisée de coupe par analyse IA du visage (forme et teinte de peau).

2. Architecture de la plateforme

2.1 Vue d'ensemble

L'architecture repose sur une séparation claire en couches : un frontend Angular, un backend NestJS exposant une API REST, un stockage hybride (MongoDB pour les métadonnées, disque pour les fichiers) et un module Python d'entraînement.

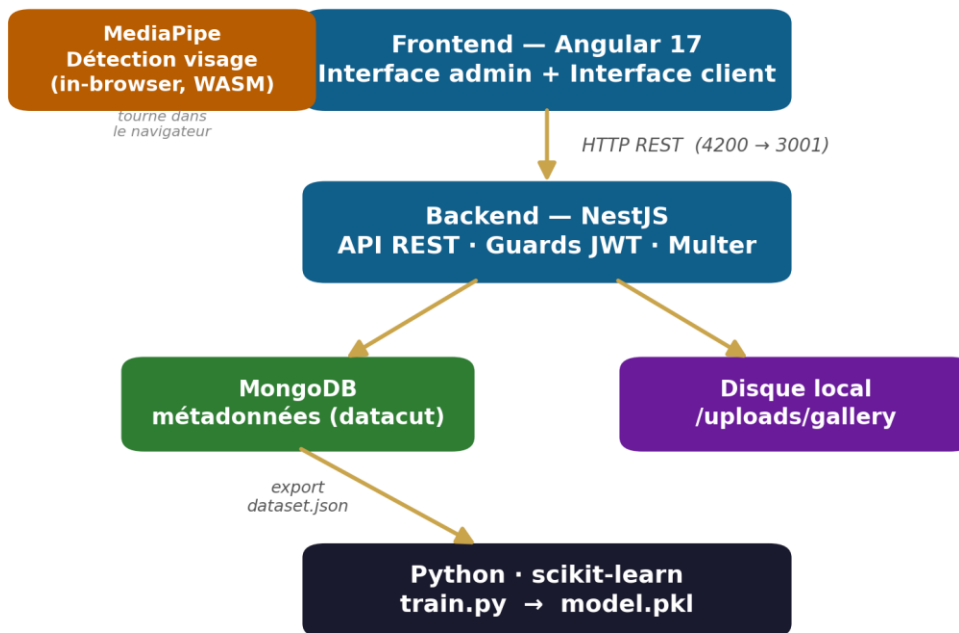


Figure 1 — Architecture en couches de la plateforme DataCut

2.2 Stack technique

Couche	Technologie	Rôle
Frontend	Angular 17 (standalone, signals)	Interface utilisateur
Backend	NestJS + Mongoose	API REST, logique métier
Base de données	MongoDB	Stockage des métadonnées
Stockage fichiers	Système de fichiers (multer)	Stockage des images brutes
Feature extraction	sharp (Node.js)	Extraction automatique de caractéristiques
ML entraînement	Python / scikit-learn	Classification multi-label
ML client	MediaPipe (JavaScript)	Détection de forme de visage in-browser

2.3 Justification des choix techniques

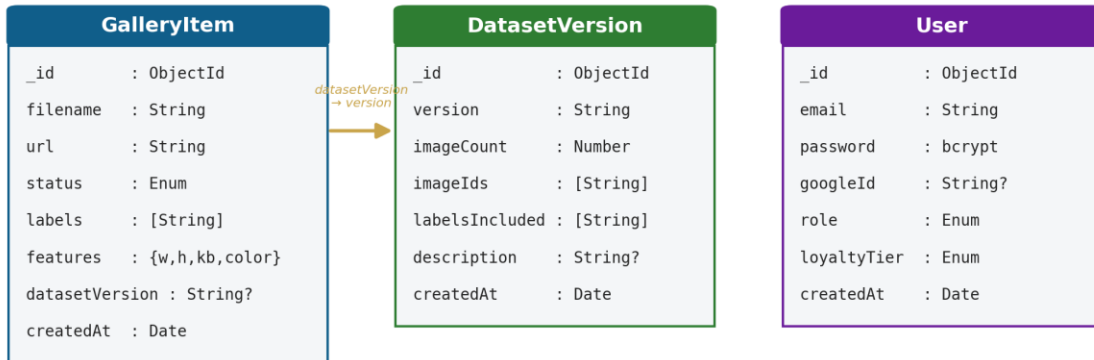
- MongoDB : flexibilité de schéma idéale pour des métadonnées hétérogènes, agrégations natives pour les statistiques de labels.

- NestJS : architecture modulaire (GalleryModule, DatasetModule, StatsModule), séparation claire des responsabilités.
- sharp : extraction de features légère côté serveur, sans dépendance externe, au moment de l'upload.
- scikit-learn : implémentation éprouvée du RandomForest multi-label, adaptée à un dataset de taille limitée.
- MediaPipe : s'exécute en WebAssembly dans le navigateur, aucune donnée biométrique transmise à un serveur externe.

3. Schéma de base de données

3.1 Principes de modélisation

La base de données datacut suit une approche hybride : les fichiers images sont stockés sur disque (/uploads/gallery/), tandis que toutes les métadonnées associées (statut, labels, features, versioning) sont stockées dans MongoDB.



Approche hybride : fichiers sur disque · métadonnées en MongoDB

Figure 2 — Modèle de données : collections et relation logique

3.2 Collection GalleryItem

```
GalleryItem {
  _id          : ObjectId      // Identifiant unique MongoDB
  filename     : String        // Nom du fichier sur disque
  url          : String        // Chemin (/uploads/gallery/...)
  category     : String?       // coupe | barbe | dégradé...
  status       : Enum          // raw|validated|labeled|processed|exported
  labels       : [String]      // Labels assignés (fade, afro, tresse...)
  features     : { width, height, sizeKb, format, dominantColor }
  datasetVersion : String?     // Version d'export
  createdAt, updatedAt : Date
}
```

3.3 Collection DatasetVersion

```
DatasetVersion {
  _id, version, imageCount, imageIds[], labelsIncluded[], description, createdAt
}
```

3.4 Relations

- `GalleryItem.datasetVersion` → `DatasetVersion.version` (clé logique, document store).

- `DatasetVersion.imageIds` liste les `GalleryItem` inclus dans le snapshot.
- Relation volontairement légère : les snapshots sont immuables.

4. Pipeline de données

4.1 Vue d'ensemble

Le pipeline structure le cycle de vie d'une image en 5 statuts successifs et unidirectionnels :

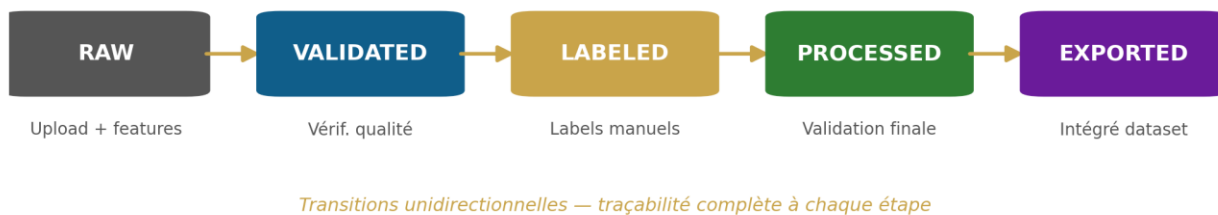


Figure 3 — Pipeline de données en 5 statuts

4.2 Description de chaque étape

Phase 1 — RAW (Ingestion)

NestJS + Multer sauvegarde le fichier, sharp extrait automatiquement les features, et un document GalleryItem est créé avec status: 'raw'.

Phase 2 — VALIDATED

L'administrateur vérifie la qualité visuelle (cadrage, netteté, pertinence) depuis l'interface Pipeline.

Phase 3 — LABELED

Assignment de 1 à N labels parmi 12 disponibles (fade, dégradé, taper, burst-fade, afro, tresse, barbe, rasage, avant, après, enfant, adulte).

Phase 4 — PROCESSED

Validation finale : l'image est complète et prête à intégrer un dataset.

Phase 5 — EXPORTED

L'image est incluse dans un snapshot versionné ; son champ datasetVersion est renseigné.

4.3 Règles de transition

- Transitions unidirectionnelles : pas de retour à un statut antérieur.
- Seules les images labeled / processed / exported sont éligibles à un dataset.
- Le funnel Pipeline visualise les volumes par statut et révèle les goulots d'étranglement.

5. Extraction de features

5.1 Principe

À chaque upload, le backend extrait automatiquement un vecteur de 7 caractéristiques numériques via sharp (Node.js). Ce vecteur constitue l'entrée du modèle ML, sans intervention humaine.

5.2 Features extraites

Index	Feature	Description	Exemple
0	width	Largeur en pixels	1080
1	height	Hauteur en pixels	1350
2	sizeKb	Taille du fichier en Ko	312.4
3	ratio	Largeur / Hauteur	0.80
4	r	Rouge de la couleur dominante	142
5	g	Vert de la couleur dominante	98
6	b	Bleu de la couleur dominante	67

5.3 Implémentation Python

```
def build_features(images):
    rows = []
    for img in images:
        feat = img.get('features') or {}
        w, h = feat.get('width',0), feat.get('height',0)
        size = feat.get('sizeKb',0)
        ratio = w / h if h > 0 else 1.0
        r,g,b = hex_to_rgb(feat.get('dominantColor','#808080'))
        rows.append([w, h, size, ratio, r, g, b])
    return np.array(rows, dtype=float)
```

6. Versioning des datasets

6.1 Principe

Le versioning repose sur des snapshots immuables : à un instant T, toutes les images éligibles sont capturées dans une version numérotée qui ne change jamais — garantissant la reproductibilité des entraînements.

6.2 Mécanisme de création

- L'administrateur clique sur « Créer une version ».
- Le backend identifie les images éligibles (labeled, processed, exported).
- Un document DatasetVersion est créé (numéro incrémental, imageIds, labelsIncluded, imageCount).
- Les images incluses passent à exported et reçoivent le tag de version.

6.3 Structure du JSON exporté

```
{
  "version": "v1", "createdAt": "2026-04-13T10:20:16Z", "imageCount": 9,
  "labelsIncluded": ["burst-fade", "taper", "barbe", "afro", "dégradé", ...],
  "images": [
    { "id": "...", "labels": ["burst-fade", "adulte", "après"],
      "features": { "width": 705, "height": 921, "sizeKb": 677, "dominantColor": "#686878" } }
  ]
}
```

7. Entraînement du modèle

7.1 Algorithme choisi

Le classificateur est un RandomForest multi-label (scikit-learn). La stratégie OneVsRestClassifier traite indépendamment chaque label : une image peut en porter plusieurs (ex : fade + barbe + adulte).

7.2 Pipeline d'entraînement

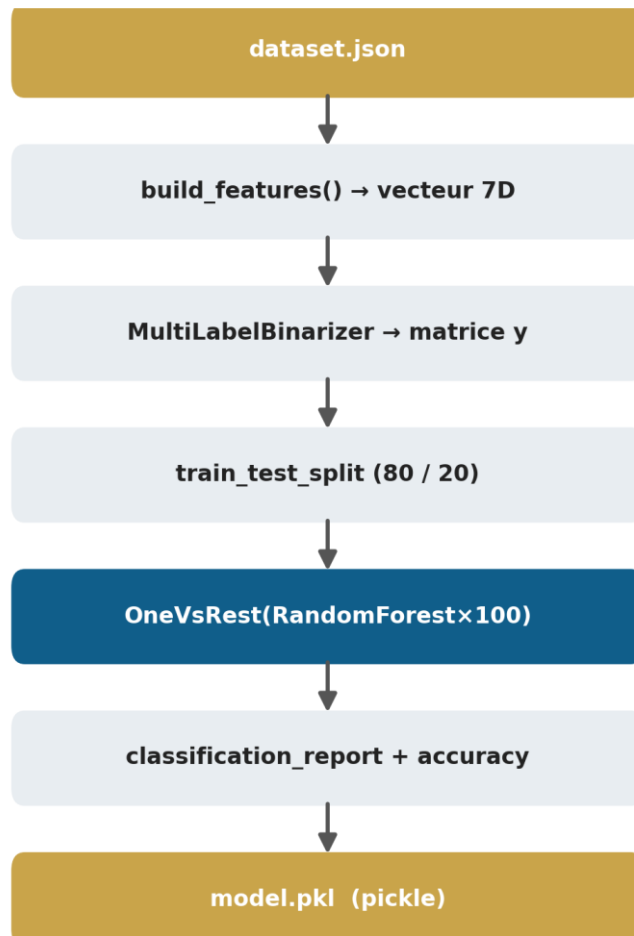


Figure 4 — Chaîne d'entraînement de train.py

7.3 Paramètres

Paramètre	Valeur	Justification
n_estimators	100	Compromis précision / vitesse
test_size	0.2	Split 80/20 standard
random_state	42	Reproductibilité
Stratégie	OneVsRest	Indépendance des labels

7.4 Utilisation du modèle

```
import pickle
model = pickle.load(open('model.pkl','rb'))
features = [[1080,1350,312.4,0.8,142,98,67]]
pred = model['clf'].predict(features)
labels = model['mlb'].inverse_transform(pred)
```

8. Visualisation & supervision

8.1 Funnel Pipeline

Un funnel interactif montre le volume d'images par statut. Chaque barre est cliquable et filtre la liste, révélant instantanément les goulots d'étranglement.

8.2 Distribution des labels

La distribution des labels permet d'identifier les classes sous-représentées — indicateur clé de la qualité du dataset. Exemple sur le dataset v1 :

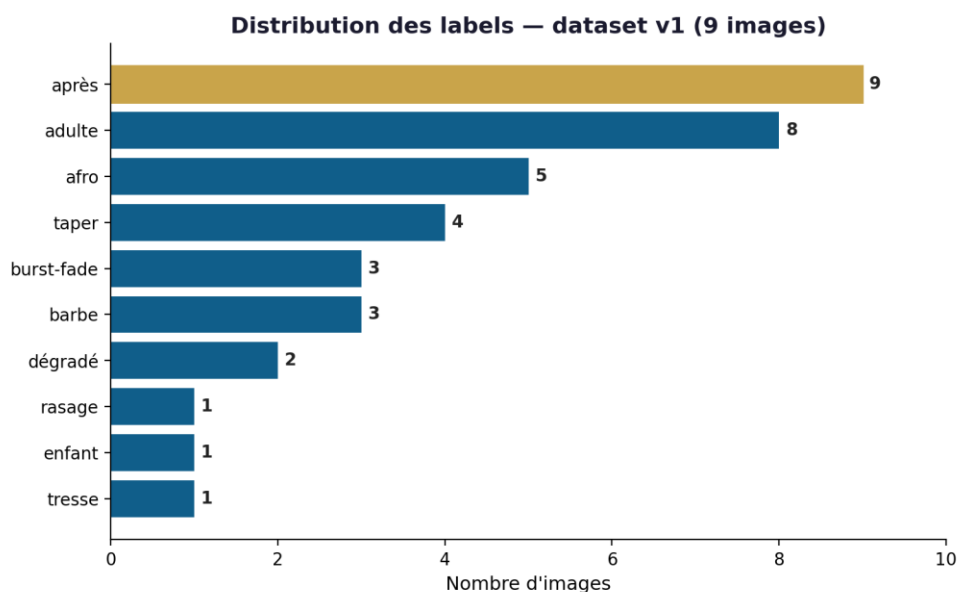


Figure 5 — Distribution réelle des labels du dataset v1

8.3 Croissance & statistiques de versions

- Graphique de croissance : nombre d'images ajoutées par semaine (8 semaines).
- Chaque version affiche : nombre d'images, labels inclus, date, export JSON direct.

9. Recommandation client par IA

9.1 Principe

La page « Ma Coupe Idéale » fournit une recommandation personnalisée à partir du visage du client. L'analyse est réalisée entièrement dans le navigateur via MediaPipe — aucune donnée n'est transmise à un serveur externe.

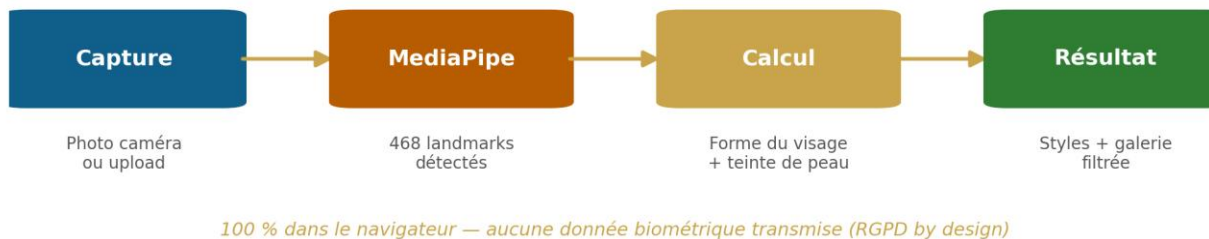


Figure 6 — Parcours de recommandation client (in-browser)

9.2 Détection de la forme du visage

MediaPipe Face Landmarker détecte 468 points de repère. Les proportions calculées :

Mesure	Landmarks	Description
Hauteur du visage	10 → 152	Front → menton
Largeur pommettes	234 → 454	Point le plus large
Largeur mâchoire	172 → 397	Ligne mandibulaire
Largeur front	70 → 300	Tempes

```
ratio    = hauteur / largeur pommettes
jawRatio = largeur mâchoire / largeur pommettes
ratio > 1.60          → Oblongue
ratio < 1.20 et jaw > 0.85 → Ronde
front > 1.08 et jaw < 0.72 → Cœur
jaw > 0.80 et ratio ≤ 1.55 → Carrée
sinon          → Ovale
```

9.3 Détection de la teinte de peau

Teinte	Luminosité HSL	Texture estimée	Styles exclus
Clair	> 58 %	Lisses à ondulés	afro, tresse
Médium	36–58 %	Ondulés à bouclés	aucun
Foncé	< 36 %	Crépus à bouclés serrés	aucun

10. Bonnes pratiques & gouvernance

10.1 Séparation fichiers / métadonnées

- Fichiers images → disque (/uploads/gallery/)
- Métadonnées → MongoDB (statut, labels, features, références)

Cette séparation permet de migrer vers S3 ou un CDN sans toucher à la base de données.

10.2 Versioning immuable

Les snapshots sont immuables. Relancer train.py sur le même JSON produit exactement le même modèle — reproductibilité et auditabilité garanties.

10.3 Sécurité & protection des données

- Authentification JWT, rôles client / admin, OAuth2 Google.
- Guards NestJS sur toutes les routes sensibles.
- MediaPipe in-browser : aucune donnée biométrique transmise (RGPD by design).

11. Conclusion & perspectives

11.1 Bilan

Exigence	Statut
Architecture détaillée de la plateforme	Réalisé
Schéma de base de données documenté	Réalisé
Pipeline de données (5 statuts)	Réalisé
Extraction de features automatique	Réalisé
Versioning immuable des datasets	Réalisé
Alimentation d'un modèle ML	Réalisé
Visualisation des flux et indicateurs	Réalisé
Démonstration fonctionnelle	Réalisé
Séparation fichiers / métadonnées	Réalisé
Recommandation client IA (bonus)	Réalisé

11.2 Perspectives d'évolution

- Dataset plus volumineux (objectif : 100+ images par label).
- Stockage cloud (AWS S3, Cloudinary).
- API de prédiction temps réel exposant model.pkl.
- Réentraînement automatique déclenché par un nouveau dataset.
- CNN / Vision Transformer pour remplacer le RandomForest.